Guide d'exploitation

La pile se lance, se sauvegarde, se restaure et change de modèle sans modifier le code. Les clés restent dans `.env`. Le mot de passe du premier administrateur est écrit plus bas : c'est le seul accès tant que d'autres comptes n'existent pas.

Installation

Prérequis : Docker, à la racine du dépôt.

```
cp .env.example .env
docker compose up --build -d
```

Puis ouvrir `http://localhost:3000`.

Service	Adresse	Rôle
Écran	`http://localhost:3000`	Les quatre rôles
API	`http://localhost:8000`	FastAPI. `GET /health` répond `status: ok`
n8n	`http://localhost:5678`	Alerte et lecture de PDF
Superset	`http://localhost:8088`	Lecture du schéma `bi`
PostgreSQL	volume `pgdata`, port non publié	Données, vecteurs, audit

Le premier passage du service `migrate` charge la cohorte synthétique, reconstruit le schéma `clean` et crée le compte administrateur `predicare` (mot de passe `predicareAdmin`). Il ne crée pas les autres rôles : ils s'ajoutent dans l'écran Utilisateurs.

Le modèle de langage est Gemma, via l'API Google (`LLM_BASE_URL`, `LLM_API_KEY`, `LLM_MODEL`). Aucun service Ollama n'est requis.

Configuration

Variable	Rôle
`ALERT_THRESHOLD`	Valeur initiale du seuil d'alerte, recopiée dans `app.settings` seulement si la clé est absente
`DRIFT_PSI_THRESHOLD`	Valeur initiale du seuil PSI de dérive, même règle
`DATABASE_URL`	Compte applicatif `prediccare_app`
`MIGRATE_DATABASE_URL`	Compte propriétaire, migrations et chargement
`BI_DB_PASSWORD`	Lecture seule du schéma `bi`
`AUTH_TOKEN_SECRET`, `ADMIN_API_KEY`, `PSEUDO_HMAC_KEY`, `PII_ENCRYPTION_KEY`	Secrets hors de la base et hors du code
`LLM_API_KEY`, `LLM_MODEL`, `LLM_BASE_URL`	Lecture des PDF et assistant

Après le premier démarrage, le seuil d'alerte, les canaux, l'adresse du workflow, la version active et le seuil de dérive se changent dans Réglages, ou par `PATCH /admin/settings`. Le fichier `.env` ne réécrit pas une clé déjà présente.

Le canal `webhook` écrit le journal interne. Slack et e-mail partent seulement si leurs adresses sont renseignées dans l'environnement du workflow et si le canal est activé. Une adresse vide journalise un échec, sans bloquer le score.

Sauvegarde

Chaque jour, dans le conteneur base :

```
docker compose exec -T db sh /backup/backup.sh
```

Le script écrit `/backup/prediccare-AAAAMMJJ.dump` et recopie le fichier vers `/backup/prediccare.dump`. Le format est `pg_dump -Fc` : données, schémas, audit et extension vectorielle. Le dossier `db/backup/` du dépôt reçoit ces fichiers. Ils ne sont pas versionnés.

Hôte Linux :

```
0 2 * * * cd /chemin/prediccare && docker compose exec -T db sh /backup/backup.sh
```

Sur Windows, la même commande se place dans le Planificateur de tâches.

Restauration

Preuve dans une base à part, sans remplacer celle qui sert l'API :

```
docker compose exec -T db sh /backup/restore.sh /backup/prediccare.dump prediccare_preuve
```

Comparer les comptes (`clean.factures`, `app.predictions`, `app.audit_log`), puis supprimer la base de preuve :

```
docker compose exec -T db dropdb -U prediccare prediccare_preuve
```

Restaurer la base qui sert l'API remplace son contenu. À réserver à un incident, après avoir arrêté l'API :

```
docker compose exec -T db sh /backup/restore.sh /backup/prediccare.dump
```

Ne pas lancer `docker volume prune` : il efface le volume `pgdata`.

Supervision

L'écran Supervision, ou `GET /admin/supervision`, rassemble le volume de scores, la latence (moyenne, p50, p95), les erreurs, la dérive et les réglages.

`GET /health` indique que l'API et la base répondent. Il ne charge pas le pipeline : une version de modèle inutilisable peut laisser la santé au vert et refuser le dépôt.

Les journaux de l'API sont des lignes JSON : heure, niveau, chemin, statut HTTP, durée. Ils ne contiennent ni le corps de la facture, ni le jeton, ni un nom.

```
docker compose logs api --tail 50
```

La dérive (PSI) ne s'alerte qu'à partir de 30 scores de la version active, et seulement si `models/<version>/metadata.json` contient `distribution_reference` (quatre parts de 0 à 1 dont la somme vaut 1). Sans cette clé, l'état reste `reference_absente`. Les fiches `v1` et `v2` livrées n'ont pas encore cette clé.

Mise à jour du modèle et retour arrière

Les poids sont `models/v1/model.joblib` et `models/v2/model.joblib`, montés dans l'API. Chaque pipeline a 23 variables, dans l'ordre de sa fiche. La version active au démarrage est `v2`.

Dans l'écran Modèles, ou :

```
PATCH /admin/settings
{"active_model_version": "v1"}
```

`GET /health` reste disponible pendant le changement. Si les colonnes du fichier ne correspondent pas à la fiche, le dépôt répond `503` et `MODELE_INDISPONIBLE`. La facture n'est pas enregistrée. On revient alors à une version qui score, puis on redépose.

Le seuil de rappel écrit sur la fiche (environ 0,54 pour `v2`, environ 0,72 pour `v1`) sert au calibrage de l'entraînement. Il n'est pas le seuil d'alerte de la plateforme.

Redémarrage

```
docker compose restart db api n8n
```

Les données sont dans `pgdata`. Après le redémarrage, `/health` répond et les factures déjà scorées sont toujours là.

L'écran est une image construite. Un changement de l'interface demande :

```
docker compose up -d --no-deps --build frontend
```

Le code Python de l'API est monté depuis `./backend/app` et rechargé par le processus.

Incidents courants

Symptôme	Action
`/health` en 503, base indisponible	`docker compose ps`, journaux de `db`, puis `docker compose restart db`. Si la base ne démarre plus, restaurer `prediccare.dump` sur une machine propre
Dépôt lent, p95 au-dessus de 500 ms	Journaux `duree_ms`. Redémarrer l'API. Si seule la version `v2` est lente, repasser à `v1` le temps du diagnostic
Facture en alerte, rien dans le journal d'envoi	Vérifier l'URL du webhook (`http://n8n:5678/webhook/alerte`, pas `localhost`), le workflow « PrediCare alerte », puis redémarrer n8n. Une URL de canal vide journalise un échec immédiat
`POST /invoices` en `MODELE_INDISPONIBLE`	Vérifier la version active et la présence du fichier de poids. Revenir à une version qui score. Les factures refusées ne sont pas insérées
Dérive `depasse: true`	Ne pas rejeter les factures à partir de ce seul signal. Lire le PSI, le seuil et l'effectif. Décider un retour de version avec la personne qui tient le modèle
Lecture de PDF en `MODELE_INDISPONIBLE`	Le workflow n8n d'OCR n'atteint pas le modèle. Vérifier `LLM_API_KEY` et `LLM_BASE_URL` dans le conteneur n8n, puis réimporter `n8n/workflows/ocr.json`

Le détail des mêmes cas est dans `docs/06-exploitation/runbook.md`.

Tests

```
pip install -r backend/requirements-dev.txt
pytest
pip-audit -r backend/requirements.txt
```

La chaîne d'intégration exécute les tests et l'audit des dépendances.